

论文《Dynamic Searchable Symmetric Encryption with Forward and Stronger Backward Privacy》

1. 研究背景

随着云存储的普及，用户通常会先对数据加密后再上传云端，从而保护数据隐私。但加密会破坏数据的可搜索性，为了解决这一矛盾，人们提出了**可搜索对称加密**（SSE）机制，使服务器能够在不知道明文的情况下对密文进行关键词检索。早期SSE方案多为**静态**的，不支持数据更新，限制了应用场景。为此，引入了**动态可搜索对称加密**（DSSE），它允许用户在加密数据上动态地插入或删除文件，同时依旧能进行检索。

然而，动态更新操作会引入新的安全泄露风险：对手可以通过观察更新操作泄露有关关键词或查询的信息。例如，向数据库中插入精心构造的文件可能暴露用户的查询（“文件注入攻击”）。为此，近年来提出了两种新的安全模型：**前向隐私**（Forward Privacy）和**后向隐私**（Backward Privacy）。前向隐私要求新加入的文件不应泄露与之前的查询相关的信息；而后向隐私要求被删除的文件不应在之后的查询中泄露过多信息。其中，Bost 等人将后向隐私细分为三种等级（Type-III 到 Type-I，表示从弱到强的安全保障）。目前，大部分研究侧重于前向隐私，而高强度的后向隐私（Type-I 及以上）仍然缺乏高效实用的方案。

2. 现有研究的不足

针对后向隐私，已有研究提出了不同级别的 DSSE 方案。但最强的 Type-I 后向隐私方案往往效率低下或不够实用。例如，Bost 等人的 MONETA 方案虽然能达到 Type-I 后向隐私，但它依赖 **TWORAM** 技术，计算开销巨大，被证明难以实际部署。Ghareh Chamani 等人也指出，TWORAM 结构下的方案主要具有理论意义，不利于实际应用。此外，Chamani 等人提出的 ORION 方案实现了 Type-I 安全，但需要 $O(\log N)$ 次交互并使用 ORAM；它的另一种方案 HORUS 虽然减少了交互轮数，但安全性退化到 Type-III，并且通信轮数与删除次数相关。其他诸如 DIANAdel、Janus、Janus++ 等方案，只能实现弱后向隐私（Type-III或Type-II），或者存在多轮交互、昂贵的密码操作等问题。综上所述，目前尚无方案在**不依赖可信硬件**的前提下，同时兼顾**强后向隐私**（Type-I）和单轮交互、低开销**的实用性需求。

3. 本文核心创新点

本文针对上述不足，提出了一种新的DSSE方案（称为FB-DSSE），其主要贡献包括：

Type-I⁻后向隐私定义：首次提出了比 Type-I 更强的后向隐私（称为Type-I⁻）。Type-I⁻ 在泄露信息上进一步减少：对于任意关键词 w ，Type-I⁻ 只泄露该关键词过去更新的次数和发生更新的**时间点**，以及当前匹配的文件，而**不泄露每个文件的具体插入时间**。这一点比原 Type-I 更严格，但仍保持了较弱的泄露（只泄露更新数量和发生时刻）。

新设计：FB-DSSE方案：本文提出了基于**位图索引**（Bitmap Index）和**同态加法对称加密**的新型DSSE方案。该方案只使用对称密钥原语，无需公钥或ORAM等昂贵结构，从而运算轻量。每个文件集合以一串二进制位（位图）表示，并用一次性对称密钥加密，同态加法支持将多次更新的密文累加。方案保持前向隐私并实现了Type-I⁻后向隐私，且在一次交互内完成检索和更新。

扩展到多块方案：为了支持**十亿级**的大规模文件集，作者将 FB-DSSE 扩展为 **多块 MB-FB-DSSE** 方案。思路是将超长位图分割为多个较短的块，每个块独立地执行 FB-DSSE 算法。这样可并行操作多块位图，在硬件资源充足时显著加快整体计算，同时保证与单块方案相同的安全性质。

效率优化：在前向隐私的实现上，本文采用了全对称的新技巧，用哈希构造而非原文献中使用的单向置换，进一步提高效率。综合来看，FB-DSSE 仅用到了轻量的哈希和模加密运算，极易部署于实际系统中。

4. 方案相较前人的优越之处

与已有方案相比，本文 FB-DSSE/MB-FB-DSSE 方案在安全性、效率和可扩展性方面均具有明显优势。**安全性**方面，本方案实现了新的 Type-I⁻ 后向隐私定义，泄露信息更少；并且依旧满足前向隐私。表1中列出了与已有工作对比，本方案仅需 1 轮交互、无需使用 ORAM 即可达到比 Type-I 更强的后向隐私。

Table 1. Comparison with previous works

Scheme	Roundtrips bet. client and server	Client storage	Forward privacy	Backward privacy	Without ORAM
FIDES [3]	2	$O(\mathbf{W} \log D)$	✓	Type-II	✓
DIANA _{del} [3]	2	$O(\mathbf{W} \log D)$	✓	Type-III	✓
Janus [3]	1	$O(\mathbf{W} \log D)$	✓	Type-III	✓
Janus++ [19]	1	$O(\mathbf{W} \log D)$	✓	Type-III	✓
MITRA [12]	2	$O(\mathbf{W} \log D)$	✓	Type-II	✓
HORUS [12]	$O(\log d_w)$	$O(\mathbf{W} \log D)$	✓	Type-III	✗
SchemeB [23]	2	$O(2 \mathbf{W} \log D)$	✗	Unknown	✓
MONETA [3]	3	$O(1)$	✓	Type-I	✗
ORION [12]	$O(\log N)$	$O(1)$	✓	Type-I	✗
Our schemes	1	$O(\mathbf{W} \log D)$	✓	Type-I ⁻	✓

N is the number of keyword/file-identifier pairs, d_w is the number of deleted entries for keyword w . $|\mathbf{W}|$ is the collection of distinct keywords, $|D|$ is the total number of files.

图1：传统依赖重型结构（如ORAM/TWORAM）的DSSE方案往往效率低下，如“蜗牛”般缓慢。本文方案只用对称加法运算，大大提升了性能。

在**效率**上，FB-DSSE 只进行常数轮交互，且服务器端主要运算是若干位图密文做简单的模加累加（见下文）。与Janus++ 需要对称加密运算、ORION 需要多轮或 ORAM 操作相比，本方案运算量极小。实验结果显示，MB-FB-DSSE 在支持 10^9 文件时（使用 10^3 块，每块长度 10^6 位），单关键词搜索耗时仅 5.84 秒，更新仅 46.41 毫秒；而未分块的 FB-DSSE 在同样规模下搜索耗时 9.07 秒，更新 125.23 毫秒。这表明多块方案在海量数据下具有更好的可扩展性和性能。此外，表3中对比了各方案的计算复杂度，FB-DSSE 的搜索复杂度是 $O(a_w)$ 次模加（ a_w 为相关更新数），而 Janus/JANUS++ 等方案需要多次对称加密解密，MONETA/ORION 等方案涉及 ORAM，计算远更重。综上，本文方案在不牺牲安全性的前提下实现了更低的信息泄露、更轻量的计算和更好的扩展性。

5. 方案设计与实现原理

本文给出的 FB-DSSE 方案基于以下关键技术：**位图索引**、**一次性密钥**和**同态加法对称加密**。我们简要说明方案的主要操作流程，分为客户端（Client）和服务器（Server）两部分，如下所示。

数据结构：客户端维护一个状态表 CT，记录每个关键词 w 的当前搜索 token ST_c 和更新计数器 c 。服务器维护一个加密数据库 EDB，用于存储以更新 token 为键的密文对。

密钥和哈希函数：客户端生成一个主密钥 K 并使用伪随机函数 F_K 对关键词 w 分出两个子密钥 $K_w \parallel K_{0,w}$ 。其中 K_w 用于生成搜索/更新 token, $K_{0,w}$ 用于派生一次性对称密钥。还使用三种哈希函数 H_1, H_2, H_3 : H_1 生成更新 token, H_2 用于“蒙版”前一个搜索 token, H_3 用于生成一次性对称密钥 sk。

同态加密：采用一种简单的一次性对称加密 Π , 其加密过程为 $c = sk + m \pmod{n}$, 解密为 $m = c - sk \pmod{n}$ 。密钥 sk 每次随机一次性使用, 累加两个密文 $c_0 + c_1$ 后解密得到消息之和。该加法性质使服务器可以将多个密文累加后返回合并结果。

更新操作 (Update) :

1. 客户端输入关键词 w 和一个位图 bs (第 $c + 1$ 次更新对应的文件集合)。首先从伪随机函数派生 $K_w, K_{0,w} = F_K(w)$; 然后从 CT 中读取 (ST_c, c) , 若无, 则初始化 $c = -1, ST_c$ 为随机比特串。
2. 客户端令 ST_{c+1} 为新的随机搜索 token, 计数器 $c \leftarrow c + 1$ 并更新 CT, 将 (ST_{c+1}, c) 写入 CT。
3. 生成**更新token**: $UT_{c+1} = H_1(K_w, ST_{c+1})$; 生成**蒙版值**: $C_{ST_c} = H_2(K_w, ST_{c+1}) \oplus ST_c$ 。该 C_{ST_c} 允许服务器在后续检索中恢复前一个 ST_c 。
4. 生成一次性对称密钥: $sk_{c+1} = H_3(K_{0,w}, c + 1)$, 并用该 sk_{c+1} 对位图 bs 加密: $e_{c+1} = \text{Enc}(sk_{c+1}, bs, n)$ 。这里 n 是公开模数, 决定系统可支持的最大文件数。
5. 客户端将更新 token UT_{c+1} 与密文对 (e_{c+1}, C_{ST_c}) 一起发送给服务器。服务器接收后将其写入 EDB: $EDB[UT_{c+1}] = (e_{c+1}, C_{ST_c})$ 。

搜索操作 (Search) :

1. 客户端输入关键词 w , 从 CT 中读出当前 (ST_c, c) , 并将 (K_w, ST_c, c) 发送给服务器。服务器现在知道需要检索的关键词对应的最新搜索 token 和更新计数。
2. 服务器端接收到 (K_w, ST_c, c) 后, 进行如下循环: 令 $S_{\text{sum}} = 0$ 。对于 $i = c, c - 1, \dots, 0$, 按如下方式逐步恢复并累加密文:
 - 服务器根据 $UT_i = H_1(K_w, ST_i)$ 查找 EDB, 取出 $(e_i, C_{ST_{i-1}})$ (其中 ST_i 在前一次循环已知, 初始为 ST_c), 并将 $S_{\text{sum}} \leftarrow S_{\text{sum}} + e_i \pmod{n}$ 。
 - 服务器删除 EDB 中已用的条目 $EDB[UT_i]$, 然后用 $C_{ST_{i-1}}$ 恢复前一次的搜索 token: $ST_{i-1} = H_2(K_w, ST_i) \oplus C_{ST_{i-1}}$ 。如果遇到 $C_{ST_{i-1}} = \perp$ (初始标记), 则退出循环。
3. 循环结束时, 服务器将累加结果 S_{sum} (对应当期所有更新的密文和) 重新存入 EDB: $EDB[UT_c] = (S_{\text{sum}}, \perp)$, 作为新的条目录录当前状态。然后服务器将 S_{sum} 返回给客户端。
4. 客户端收到 S_{sum} 后, 用同样的生成规则累加出所有一次性密钥之和: $\text{Sum} * sk = \sum_{i=0}^c sk_i \pmod{n}$ 。然后用 $\text{Dec}(\text{Sum} * sk, S * \text{sum}, n)$ 解密即可恢复位图 $bs = \sum_{i=0}^c bs_i$, 其中每个 bs_i 对应一次更新中的文件集合。这一位图表示满足关键词 w 的所有活跃文件 ID; 客户端据此检索相应文件。

上述流程说明: 服务器通过**同态累加**将所有相关更新的密文合并为一个密文 S_{sum} , 客户端只需一次解密操作即可获取所有匹配文件, 无需逐条解密。每次搜索后, 服务器删除所有旧条目, 仅保留累加后的最终密文, 从而**隐藏了各个更新对应的具体信息**, 达到了后向隐私要求。

多块扩展 (MB-FB-DSSE) : 在上述流程基础上, 将位图 bs 分解为 B 个子块 $bs[1], \dots, bs[B]$, 每块长度约为 ℓ/B (ℓ 是总位长)。对于每块, 各自执行与单块方案相同的更新和搜索协议。也就是说, 客户端会为每块生成独立的密文 $e[j] = \text{Enc}(sk[j], bs[j], n)$ 并更新到对应的 UT 中, 服务器在搜索时对每块分别累加并返回 $S_{\text{sum}}[j]$ 。客户端解密后将各块结果拼合得到完整位图。这样多块并行计算显著提升了大数据集下的效率。

6. 主要结果与结论

本文给出 FB-DSSE 和 MB-FB-DSSE 的安全证明和实验评估，结果表明方案既安全又高效。安全性方面，通过严格的形式化分析证明：FB-DSSE 方案满足前向隐私，并且满足新的 Type-I⁻ 后向隐私定义；MB-FB-DSSE 在每块上同样保证这一安全性。也就是说，在任何时刻被删除的文件对应的信息（除了更新次数和时间点）均不会泄露。

性能评估中，作者实现了原型并在单机上测试。表1所列实验结果显示，FB-DSSE 对 10^9 文件的单关键字搜索仅需约 9.07 秒、更新约 125.23 毫秒。采用多块（MB-FB-DSSE）后，搜索加速至约 5.84 秒，更新 46.41 毫秒。这些结果证明方案支持海量文件、检索和更新延迟很低。总体而言，本文方案在保证更强后向隐私的同时，计算开销和交互轮数都大大优于已有工作。

结论：本文提出了一种基于对称加密与位图索引的新型 DSSE 方案，实现了前向隐私及比 Type-I 更强的后向隐私（Type-I⁻），且仅需单轮交互、无重型结构依赖，具有极高的实用价值。扩展到多块后，方案可伸缩到上十亿文件规模。实验证明其搜索/更新性能优异，安全性由理论证明支持。未来，本文方案可作为高效安全可搜索加密系统的基础，为云端数据检索和管理提供更强隐私保障。

Type-I 后向安全泄漏了有关何时插入包含 w 的新文件以及 w 上更新的总数的信息

Type-II 后向安全泄漏了关于关键词 w 的所有更新（包括删除）发生的时间

Type-III 后向安全泄漏了哪次更新取消了哪次删除

前向隐私 (Forward Privacy)

- **核心理念：更新操作**（如添加 / 删除文件）仅泄露被修改文件的标识符 f_i 和对应的关键词数量 μ_i ，而不泄露其他信息（如具体关键词内容、历史操作顺序等）。

后向隐私解决的是“被删除文件”在后续搜索中不被泄露的问题。

文章提出了一种比 Type-I 更强的后向隐私（称为 Type-I⁻），比起 I 型后向安全，Type-I⁻ 不会泄露文件的更新时间。

作者的构造

概述

已有的穿孔加密方法：

- 通过对单个文件标识符打“洞”失效来防止被删文件被搜出，但只能做到 Type-III 后向隐私；
- 而且每次“单个标识符”的插入/删除，都要做加密/打洞操作，服务器会记录下“插入/删除时间”以及对应“哪一个文件”，这正好符合 Type-III 的“全部更新时刻+哪次删除对应哪次插入”的泄露模式。

本文的新方案：

- 用**位图索引**一次性把所有文件标识符整合到一个 ℓ -位长的比特串上；插入/删除都是“对比特串的某一位做 ± 1 （模运算）”，服务器只看到“同态加密后”：一串密文里的“整体加法结果”，它不知道究竟是哪一个比特位被加或减。
- 需要用到“加法同态加密”来让服务器无须解密即可完成“累加”操作。Paillier 虽然能，但效率极低且不适用于超长位图。

- 因此改用“**一次性对称密钥+简单模加**”的方案：每次更新时都使用一个一次性密钥 sk ，把位图 bs 加上去得到密文 $c = sk + bs \bmod n$ 。这样服务器只要把所有与 w 相关的密文都逐个“+”起来，就可得到总和。客户端在最后拿到总密文时，用对应的“所有一次性密钥之和”去解密，就能还原明文位图。
- 为了节省客户端存储，把“一次性密钥”改为通过哈希函数 $H_3(K_w, i)$ 逐次生成，这样只需保存“当前更新计数器”即可按需重建全部一次性密钥。

最终效果：

- 服务器永远**只看见**“同态累加的加密结果”与“当前累加后的一条密文”，它**不知道**这过程中“是插入哪个文件的那一位”，“或者删除哪个文件的那一位”，从而即可实现更强的后向隐私（本文定义的 Type-I'）。
- 同时，客户端/服务器间仅需一轮交互，且加解密操作都是“简单模加/减”，避免了 Paillier 那种大整数模幂运算的复杂度，在海量文件场景下性能大幅提升。

作者如何使用位图索引构建方案？

使用位图索引来表示文件标识符，先设置一个长度为 ℓ 的位串 bs ，其中长度 ℓ 就是能添加的最大文件数，如果存在文件 f_i ，就在位串的第 i 位设为 1，否则设为 0。假设 $\ell = 6$ 且已经存在文件 f_2, f_3 ，那么位串表示为 001100，如果我们要添加文件 f_1 ，只需要生成位串 000010，添加到原始位串中。如果我们想删除文件 f_2 ，需要生成位串 -000100，因为要保证结果始终是 6 位，因此加法要在模 2^6 下进行，那么位串 $-000100 = -2^2 = 2^6 - 2^2 = 60 = 111100$ ，然后和原始位串相加，最终得到结果 1001000，模 2^6 后得 001000，这样就通过模加的方式操作位图索引的添加和删除。

$$60 + 12 \bmod 72 = 64 + 8$$

具有前向和更强后向隐私的 DSSE

FB-DSSE

基于 16 年前向安全的框架。并且因为位图存储在服务器端，因此不能直接使用明文进行模加操作，这里采用线性的同态加密后做加法。该方案被定义为以下三个算法：

$$1. (EDB, \sigma = (n, K, CT)) \leftarrow \text{Setup}(1^\lambda)$$

CT 是一个存储 W 中每个关键字的当前搜索令牌 ST_c 和计数器 c 的映射

$$2. (\sigma', EDB') \leftarrow \text{Update}(w, bs, \sigma; EDB)$$

客户端先将要上传的位串 bs 利用同态加法的简单对称加密得到加密比特串 e ，即 $e = sk + bs \bmod n$ ，每次更新都需要使用一个新的密钥 sk ，为了节省客户端存储，这个密钥 sk 是由哈希函数计算出来的。接下来客户端选择一个随机比特串作为搜索令牌（Search token），再对其哈希得到更新令牌（Update token）。客户端还需要用另一个哈希函数掩盖之前的搜索令牌。最终，客户端将这一次的更新令牌，加密比特串和掩盖后的搜索令牌发送给服务器并更新本地映射 $CT[w] \leftarrow (ST_{c+1}, c + 1)$ 。

$$3. bs \leftarrow \text{Search}(w, \sigma; EDB)$$

客户端从 CT 中取得关于 w 的搜索令牌 ST_c ，将其发送给服务器。

服务器对其使用哈希函数计算出更新令牌 UT ，用更新令牌取得之前存储的加密比特串 e 和掩码 $C_{ST_{i-1}}$ ，通过哈希当前的搜索令牌 ST_i 并与当前的掩码异或后得到上一个搜索令牌，继续重复上述搜索步骤，通过使用简单对称加密的同态加法（Add）将它们加在一起，得到最终结果 Sum_e 并将其发送给客户端。为了节省服务器存储空间，对于每次搜索，服务器可以删除与 w 对应的所有条目，并将与当前搜索令牌 ST_c 对应的最终结果 Sum_e 存储到 EDB。

客户端收到 Sum_e 后, 通过哈希函数计算每次更新使用的密钥 sk , 求和得到 Sum_{sk} 后对 Sum_e 解密即可得到搜索结果。

Algorithm 1. FB-DSSE

Setup(1^λ)

```

1:  $K \xleftarrow{\$} \{0, 1\}^\lambda, n \leftarrow \text{Setup}(1^\lambda)$ 
2:  $\text{CT}, \text{EDB} \leftarrow \text{empty map}$ 
3: return  $(\text{EDB}, \sigma = (n, K, \text{CT}))$ 

```

Update($w, bs, \sigma; \text{EDB}$)

Client:

```

1:  $K_w || K'_w \leftarrow F_K(w), (ST_c, c) \leftarrow \text{CT}[w]$ 
2: if  $(ST_c, c) = \perp$  then
3:    $c \leftarrow -1, ST_c \leftarrow \{0, 1\}^\lambda$ 
4: end if
5:  $ST_{c+1} \leftarrow \{0, 1\}^\lambda$ 
6:  $\text{CT}[w] \leftarrow (ST_{c+1}, c + 1)$ 
7:  $UT_{c+1} \leftarrow H_1(K_w, ST_{c+1})$ 
8:  $C_{ST_c} \leftarrow H_2(K_w, ST_{c+1}) \oplus ST_c$ 
9:  $sk_{c+1} \leftarrow H_3(K'_w, c + 1)$ 
10:  $e_{c+1} \leftarrow \text{Enc}(sk_{c+1}, bs, n)$ 
11: Send  $(UT_{c+1}, (e_{c+1}, C_{ST_c}))$  to server.

```

Server:

```

12:  $\text{EDB}[UT_{c+1}] \leftarrow (e_{c+1}, C_{ST_c})$ 

```

Search($w, \sigma; \text{EDB}$)

Client:

```

1:  $K_w || K'_w \leftarrow F_K(w), (ST_c, c) \leftarrow \text{CT}[w]$ 
2: if  $(ST_c, c) = \perp$  then
3:   return  $\emptyset$ 

```

```

4: end if

```

```

5: Send  $(K_w, ST_c, c)$  to server.

```

Server:

```

6:  $Sum_e \leftarrow 0$ 
7: for  $i = c$  to 0 do
8:    $UT_i \leftarrow H_1(K_w, ST_i)$ 
9:    $(e_i, C_{ST_{i-1}}) \leftarrow \text{EDB}[UT_i]$ 
10:   $Sum_e \leftarrow \text{Add}(Sum_e, e_i, n)$ 
11:  Remove  $\text{EDB}[UT_i]$ 
12:  if  $C_{ST_{i-1}} = \perp$  then
13:    Break
14:  end if
15:   $ST_{i-1} \leftarrow H_2(K_w, ST_i) \oplus C_{ST_{i-1}}$ 
16: end for
17:  $\text{EDB}[UT_c] \leftarrow (Sum_e, \perp)$ 
18: Send  $Sum_e$  to client.

```

Client:

```

19:  $Sum_{sk} \leftarrow 0$ 
20: for  $i = c$  to 0 do
21:    $sk_i \leftarrow H_3(K'_w, i)$ 
22:    $Sum_{sk} \leftarrow Sum_{sk} + sk_i \bmod n$ 
23: end for
24:  $bs \leftarrow \text{Dec}(Sum_{sk}, Sum_e, n)$ 
25: return  $bs$ 

```

大量文件的多块扩展——MB-FB-DSSE

理论上说, FB-DSSE 可以支持任意长度的 ℓ , 但更大的 n (例如 $= 2^{23}$) 将显著减缓模块化操作。多块拓展的想法就是将长比特序列拆分为多个较小的块, 并具有多个 (例如 B) 块 bs , 而不是一个块 bs 。对于每个 bs 块, 操作与 FB-DSSE 完全相同。

bs 是长度为 B 的位串数组

Algorithm 2. Multi-block extension MB-FB-DSSE (Difference in red)

Setup(1^λ)

1: Same as the one in FB-DSSE.

Update($w, \text{bs}, \sigma; \text{EDB}$)

Client:

1: Same as the one in FB-DSSE.

2: for $j = 0$ to B do

3: $\text{sk}_{c+1}[j] \leftarrow H_3(K'_w, c+1||j)$

4: $\text{e}_{c+1}[j] \leftarrow \text{Enc}(\text{sk}_{c+1}[j], \text{bs}[j], n)$

5: end for

6: Send $(ST_{c+1}, (\text{e}_{c+1}, C_{ST_c}))$ to the server.

Server:

7: $\text{EDB}[ST_{c+1}] \leftarrow (\text{e}_{c+1}, C_{ST_c})$

Search($w, \sigma; \text{EDB}$)

Client:

1: Same as the one in FB-DSSE.

Server:

2: $\text{Sum}_e \leftarrow 0$

3: for $i = c$ to $0, j = 0$ to B do

4: $(\text{e}_i, C_{ST_{i-1}}) \leftarrow \text{EDB}[ST_i]$

5: $\text{Sum}_e[j] \leftarrow \text{Add}(\text{Sum}_e[j], \text{e}_i[j], n)$

6: Same as the one in FB-DSSE.

7: end for

8: $\text{EDB}[ST_c] \leftarrow (\text{Sum}_e, \perp)$

9: Send Sum_e to the client.

Client:

10: $\text{Sum}_{\text{sk}} \leftarrow 0$

11: for $i = c$ to $0, j = 0$ to B do

12: $\text{sk}_i[j] \leftarrow H_3(K'_w, i||j)$

13: $\text{Sum}_{\text{sk}}[j] \leftarrow \text{Sum}_{\text{sk}}[j] + \text{sk}_i[j] \bmod n$

14: end for

15: for $j = 0$ to B do

16: $\text{bs}[j] \leftarrow \text{Dec}(\text{Sum}_{\text{sk}}[j], \text{Sum}_e[j], n)$

17: end for

18: return bs

$\text{e}, \text{sk}, \text{Sum}_e$ 和 Sum_{sk} 是长度为 B 的数组。 Sum_e 和 Sum_{sk} 分别用于存储所有加密比特串数组 e 和密钥 sk 的总和。